

1. Purpose of This Test Class

This file tests the **ProductService** class **in isolation**.

Key idea:

- We are **NOT testing the database**
- We are **NOT testing the repository implementation**
- We are testing **only the business logic inside ProductService**

To achieve this isolation, we use:

- **NUnit** → testing framework
 - **Moq** → to create fake (mocked) repository behavior
-

2. Using Statements

```
using Moq;

using NUnit.Framework;

using ProductOrderApi.DTOs;
using ProductOrderApi.Models;
using ProductOrderApi.Repositories;
using ProductOrderApi.Services;

using System.Collections.Generic;

using System.Linq;

using System.Threading.Tasks;
```

Show more lines

What these are used for:

Namespace	Purpose
Moq	Create mock objects (Mock<IRepository<Product>>)
NUnit.Framework	Test attributes and assertions
DTOs	Data Transfer Objects (CreateProductDto)

Models	Entity class (Product)
Repositories	Repository interface (IRepository<T>)
Services	Class under test (ProductService)
System.Threading.Tasks	Async/await support

3. Namespace Declaration

```
namespace ProductOrderApi.Tests.Services;
```

- ✓ Keeps test code separate from production code
 - ✓ Common convention: .Tests project
-

4. Test Class Declaration

```
[TestFixture]
```

```
public class ProductServiceTests
```

Show more lines

What [TestFixture] does:

- Marks this class as a **test container**
 - NUnit will scan this class and execute its test methods
-

5. Fields in the Test Class

```
private Mock<IRepository<Product>> _mockRepo;
```

```
private ProductService _service;
```

What these mean:

_mockRepo

- A **fake repository**
- Simulates database operations
- Allows you to control return values and verify calls

_service

- The **actual class under test**
 - Uses _mockRepo.Object instead of a real repository
-

6. Setup Method (Runs Before Each Test)

[SetUp]

```
public void Setup()
{
    _mockRepo = new Mock<IRepository<Product>>();
    _service = new ProductService(_mockRepo.Object);
}
```

What happens step-by-step:

1. [SetUp] runs **before every test method**
2. A **new mock repository** is created
3. A new ProductService is created
4. The service receives the **mocked repository**

This ensures **test isolation** & No test affects another test

7. Test 1: GetProductByIdAsync – Existing Product

[Test]

```
public async Task GetProductByIdAsync_ExistingId_ReturnsProductDto()
```

Arrange

```
int id = 1;
```

```
var product = new Product { Id = id, Name = "Laptop", Price = 1200, Stock = 5 };
```

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync(product);
```

- Create a fake Product
 - Tell the mock repository:
 - “If GetByIdAsync(1) is called, return this product”
-

Act

```
var result = await _service.GetProductByIdAsync(id);
```

- Calls the **real service method**
 - Service internally calls the mocked repository
-

Assert

```
Assert.That(result, Is.Not.Null);
```

```
Assert.That(result.Id, Is.EqualTo(id));
```

```
Assert.That(result.Name, Is.EqualTo("Laptop"));
```

```
Assert.That(result.Price, Is.EqualTo(1200));
```

```
Assert.That(result.Stock, Is.EqualTo(5));
```

Confirms:

- DTO is returned
- Mapping from Product → ProductDto is correct

8. Test 2: GetProductByIdAsync – Non-Existing Product

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync((Product)null);
```

Show more lines

- Repository returns null

```
var result = await _service.GetProductByIdAsync(id);
```

```
Assert.That(result, Is.Null);
```

Confirms:

- Service correctly handles missing data
- No exception is thrown
- Returns null

9. Test 3: GetAllProductsAsync

Arrange

```
var products = new List<Product>
```

```
{
```

```
    new Product { Id = 1, Name = "A", Price = 10, Stock = 1 },
```

```
    new Product { Id = 2, Name = "B", Price = 20, Stock = 2 }
```

```
};
```

```
_mockRepo.Setup(r => r.GetAllAsync()).ReturnsAsync(products);
```

- Mock returns a list of products
-

Act

```
var result = (await _service.GetAllProductsAsync()).ToList();
```

- Service converts entities → DTOs
-

Assert

```
Assert.That(result, Has.Count.EqualTo(2));
```

```
Assert.That(result[0].Name, Is.EqualTo("A"));
```

```
Assert.That(result[1].Name, Is.EqualTo("B"));
```

Confirms:

- Correct count
 - Correct data mapping
-

10. Test 4: CreateProductAsync

Arrange

```
var createDto = new CreateProductDto { Name = "New", Price = 100, Stock = 10 };
```

```
Product? capturedProduct = null;
```

- DTO represents client input
 - capturedProduct will capture what is sent to the repository
-

Mock Setup

```
_mockRepo.Setup(r => r.AddAsync(It.IsAny<Product>()))
```

```
.Callback<Product>(p => capturedProduct = p)
```

```
.Returns(Task.CompletedTask);
```

```
_mockRepo.Setup(r => r.SaveChangesAsync()).Returns(Task.CompletedTask);
```

- Capture the product passed to AddAsync
 - Simulate successful save
-

Act

```
var result = await _service.CreateProductAsync(createDto);
```

Assert

```
Assert.That(result.Name, Is.EqualTo("New"));
```

```
Assert.That(result.Price, Is.EqualTo(100));
```

```
Assert.That(result.Stock, Is.EqualTo(10));
```

Confirms returned DTO is correct

```
Assert.That(capturedProduct!.Name, Is.EqualTo("New"));
```

Confirms entity sent to repository is correct

```
_mockRepo.Verify(r => r.AddAsync(It.IsAny<Product>()), Times.Once);  
_mockRepo.Verify(r => r.SaveChangesAsync(), Times.Once);
```

Confirms correct repository calls

11. Test 5: UpdateProductAsync – Existing Product

Arrange

```
var existingProduct = new Product { Id = id, Name = "Old", Price = 50, Stock = 2 };  
var updateDto = new CreateProductDto { Name = "Updated", Price = 60, Stock = 3 };
```

- Mock existing product

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync(existingProduct);
```

Act

```
var result = await _service.UpdateProductAsync(id, updateDto);
```

Assert

```
Assert.That(result.Name, Is.EqualTo("Updated"));
```

DTO updated correctly

```
Assert.That(existingProduct.Name, Is.EqualTo("Updated"));
```

Entity updated correctly

```
_mockRepo.Verify(r => r.Update(existingProduct), Times.Once);  
_mockRepo.Verify(r => r.SaveChangesAsync(), Times.Once);
```

Confirms update was persisted

12. Test 6: UpdateProductAsync – Non-Existing Product

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync((Product)null);
```

```
var result = await _service.UpdateProductAsync(id, updateDto);
```

```
Assert.That(result, Is.Null);
```

Confirms safe handling of missing data

```
_mockRepo.Verify(r => r.Update(It.IsAny<Product>()), Times.Never);
```

No accidental update

13. Test 7: DeleteProductAsync – Existing Product

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync(product);
```

```
var result = await _service.DeleteProductAsync(id);
```

```
Assert.That(result, Is.True);
```

Confirms deletion success

```
_mockRepo.Verify(r => r.Delete(product), Times.Once);
```

```
_mockRepo.Verify(r => r.SaveChangesAsync(), Times.Once);
```

Confirms repository interaction

14. Test 8: DeleteProductAsync – Non-Existing Product

```
_mockRepo.Setup(r => r.GetByIdAsync(id)).ReturnsAsync((Product)null);
```

```
var result = await _service.DeleteProductAsync(id);
```

```
Assert.That(result, Is.False);
```

Confirms failure handled gracefully

```
_mockRepo.Verify(r => r.Delete(It.IsAny<Product>()), Times.Never);
```

No unintended deletion

Final Summary

This test class ensures that:

ProductService

- Correctly handles **CRUD operations**
- Correctly maps **Entities ↔ DTOs**
- Safely handles **null / missing data**
- Calls the repository **exactly when required**

Moq is used to:

- Control repository behavior
- Capture method arguments
- Verify method calls

NUnit is used to:

- Structure tests
- Assert expected outcomes